

SUPPONIAMO DI UTILIZZARE UN FILE SYSTEM UNIX BASATO SU I-NODE
 PARTICOLARI CHE CONTENGONO I SEGUENTI CAMPI: 12 INDIRIZZI DIRETTI A BLOCCHI
 DI DATI, 1 INDIRIZZO AD UN BLOCCO INDIRETTO SINGOLO E 1 INDIRIZZO AD UN
 BLOCCO INDIRETTO DOPPIO.
 SUPPONENDO DI AVERE NUMERI DI BLOCCHI A 32 BIT E BLOCCHI SU DISCO DA 1Kb,
 INDICARE ESATTAMENTE LA DIMENSIONE MASSIMA IN Kb SUPPORTATO DA UN SIME I-NODE.
 ESPlicitARE IL CALCOLO UTILIZZATO.

SOLUZIONE

- INDIRIZZI DIRETTI = 12 BLOCCHI

- BLOCCO INDIRETTO SINGOLO:

- OGNI BLOCCO INDIRETTO SINGOLO CONTIENE $1Kb / 4 \text{ byte} = 256 \text{ BLOCCHI}$

- BLOCCO INDIRETTO DOPPIO $\frac{1}{2} (1Kb / 4)^2 = (256)^2 \text{ BLOCCHI}$

- DIMENSIONE MASSIMA = $(12 + 256 + 256^2) = 65804 \text{ BLOCCHI}$

- IN BYTE $65804 \cdot 1024 \text{ Byte}$

- IN Kb $= 65804 \text{ Kb} \rightarrow 65804 \cdot 1K\text{Byte}$

$32/8 = 4 \text{ byte}$ conversione
 de bit a
 byte

è un puntatore
 ad un blocco
 di indirizzi

è l'insieme delle
 domande: quanti
 indirizzi ci sono

SI CONSIDERI UN SISTEMA CHE UTILIZZA TABELLE DELLE PAGINE A DUE LIVELLI CON DIMENSIONE DELLE PAGINE DI 4KB E INDIRIZZI A 32 BIT SUDDIVISI IN TRE CAMPI DA RISPETTIVAMENTE 10, 10, 12 (12=offset). CALCOLARE LA DIMENSIONE DELLA TABELLA DELLE PAGINE DI UN PROCESSO CHE HA ALLOCATO LE SEGUENTI AREE DI MEMORIA LOGICA:

AREA 1: DA 0 A 1.000.000

AREA 2: LOCAZIONI DA 4.200.000 A 5.200.000

SOLUZIONE

AVENDO INDIRIZZI A 32 BIT AVREMO UNA TABELLA DELLE PAGINE COSI' COMPOSTA:

TABELLA DI PRIMO LIVELLO:

$$\frac{2^{32}}{2^{12}} = 2^{20} \text{ voci}$$

PAGINA
DA
4 KB

↳ de tutto el indirizz
toli e' offset e lei tutte le voci

TABELLA
DELLE
PAGINE

TABELLA DI SECONDO LIVELLO: → punte ad une pagine con solo indirizzi

quanti indirizzi contiene $\frac{4 \text{ Kb}}{32 \text{ bit}} = \frac{4096}{4 \text{ byte}} = 1024 \text{ voci}$

↓
SIGNIFICA CHE OGM TABELLA
POTRA' OSPITARE 1024 voci

ESSENDO SIA LA PRIMA SIA LA SECONDA AREA DA POCO MEGLIO DI UN KB
1° (1.000.000 ≈ 1Kb) 2° (5.200.000 - 4.200.000 ≈ 1Kb) OCCUPERANNO

$$\frac{1.000.000}{4.096} = 244,1 \text{ voci ovvero } 245 \text{ voci}$$

↓
DIM
SINGOLA PAGINA

~~VER~~ VERRANNO ASSOCIATE INSIEME NELLA PRIMA TABELLA DI 2° LIVELLO
AVENDO $245 \times 2 < 1024$

AL PROCESSO PER SEGNARSI DOVE SI TROVANO LE SEGUENTI DUE
AREE NECESSITA DI 8 Kb

- 4 Kb PUNTATORE ALLA DIRECTORY
- 4 Kb PUNTATORE ALLA PAGE TABLE

TRA GLI ALGORITMI DI SOSTITUZIONE DELLE PAGINE ABBIAMO VISTO L'ALGORITMO DI AGING. SUPPONIAMO DI AVERE 5 PAGINE IN MEMORIA E CHE A TEMPO t LO STATO DEI CONTATORI BINARI SIA IL SEGUENTE

PAGINA	CONTATORE
A	00101000
B	10000001
C	01100000
D	00101011
E	00000010

AGING

A TEMPO $t+1$ (A:1, B:0, C:1, D:0, E:1), A TEMPO $t+2$ (A:0, B:1, C:0, D:0, E:1)
RIPORTARE QUALE PAGINA VERREBBE SOSTITUITA A TEMPO $t+3$

$t=0$

A	0	0	1	0	1	0	0	0
B	1	0	0	0	0	0	0	1
C	0	1	1	0	0	0	0	0
D	0	0	1	0	1	0	1	1
E	0	0	0	0	0	0	1	0

SHIFT A DESTRA
e aggiungo R e x

$t+1$

1	0	0	1	0	1	0	0	
0	1	0	0	0	0	0	0	
1	0	1	1	0	0	0	0	
0	0	0	1	0	1	0	1	
1	0	0	0	0	0	0	1	

SHIFT A DESTRA
e aggiungo R e x

$t+2$

A	0	1	0	0	1	0	1	0
B	1	0	1	0	0	0	0	0
C	0	1	0	1	1	0	0	0
D	0	0	0	0	1	0	1	0
E	1	1	0	0	0	0	0	0

quella con più
zeri all'inizio

84 22

A = 74, B = 160, C = 88, D = 10, E = 192

LA PAGINA CHE VERREBBE SOSTITUITA E' LA D ESSENDO LA PIU' GIOVANE

IN UN FILE SYSTEM UNIX DOVE OGNI I-NODE OCCUPA 1 BLOCCO, SI CONSIDERI 1 FILE /usr/carlo/documenti/compiro. CALCOLARE IL NUMERO DI ACCESSI AL DISCO NECESSARI PER APRIRE QUESTO FILE, SUPPONENDO CHE OGNI CARTELLA DI QUESTA PATH OCCUPI 1 BLOCCO E CHE L'I-NODE DELLA CARTELLA RADICE SIA CARICATA IN MEMORIA, MENTRE TUTTI GLI ALTRI I-NODE E TUTTE LE CARTELLE INTERESSATE RISIEDONO SU DISCO.

SOLUZIONE

FILE SYSTEM

~~1-NODE /~~ → ~~DATA BLOCK~~ → 1-NODE¹ /usr → DATA BLOCK² /usr →
 1-NODE³ /usr/carlo → DATA BLOCK⁴ /usr/carlo → 1-NODE⁵ /usr/carlo/documenti
 → DATA BLOCK⁶ /usr/carlo/documenti → 1-NODE⁷ /usr/carlo/documenti/compiro
 → DATA BLOCK⁸ /usr/carlo/documenti/compiro

- 8 LETTURE

inode → data block → inode directory successive { ogni volta che viene richiesto }
 { ogni volta che viene richiesto }

SUPPONIAMO DI AVERE 4 DISCHI DA 1 GB IMPIEGATI PER REALIZZARE UN VOLUME RAID-5. AVENDO COME RIFERIMENTO LA SEGUENTE VISIONE PARZIALE DEL CONTENUTO DEI DISCHI, RICOSTRUIRE IL CONTENUTO DEI BLOCCHI MANCANTI (X,Y) ESPlicitANDO IL CALCOLO EFFETTUATO. INDICARE INOLTRE LA CAPACITA' ATTESA (IN GB) DEL VOLUME RAID-5 COSI' OTTENUTO.

DISCO 0	DISCO 1	DISCO 3	DISCO 2
1 0 0 0 1 1 1 1	0 0 1 1 1 0 1 1	1 0 1 0 0 1 1 1	0 0 0 1 0 0 1 1
1 1 0 0 0 1 1 0	0 1 1 0 1 1 0 1	Y	1 1 1 1 0 1 0 0
1 1 0 1 0 1 0 0	0 0 1 1 1 0 1 0	1 0 1 0 1 0 0 1	0 1 0 0 0 1 1 1
0 1 0 1 0 1 1 1	1 1 1 0 0 0 1 1	0 0 0 0 1 1 0 1	1 0 1 1 1 0 0 1
...	...	...	...
...	...	...	...
...	...	...	...
X	0 1 0 1 1 0 0 1	0 1 0 0 1 1 1 1	1 0 1 1 1 1 0 1

NOTA: IN QUESTO ESEMPIO ASSUMIAMO CHE IL BLOCCO DI BASE (STRIPE) ABBA DIMENSIONE PARI A 8 BIT.

SOLUZIONE

- CONTENUTO X: XOR

$$X = 01011001 \oplus 01001111 \oplus 10111101$$

$$X = 00010110 \oplus 10111101$$

$$X = 10101011$$

- CONTENUTO Y:

$$Y = 11000110 \oplus 01101101 \oplus 11110100$$

$$Y = 10101011 \oplus 11110100$$

$$Y = 01011111$$

$$11 = 0$$

$$10 = 1$$

$$01 = 1$$

$$00 = 0$$

→ Consumi 1 slice anche se diviso tra tutti i dischi per i bit XOR

$$\text{CAPACITA' TOTALE} = (n-1) * \text{DIMENSIONE DISCO} = (4-1) * 1 \text{ GB} = 3 \text{ GB}$$

CONSIDERIAMO UN SISTEMA DI GESTIONE DELLA MEMORIA PAGINATA: ^{SUPPONENDO DI} DI AVERE 4 FRATTE. IN MEMORIA DA GESTIRE (INIZIALMENTE LIBERI), CONSIDERIAMO LA SEGUENTE SEQUENZA DI ACCESSI ALLE PAGINE:

5, 7, 7, 9, 1, 5, 3, 9, 9, 7, 2, 5

SUPPONENDO DI UTILIZZARE L'ALGORITMO LRU PER LA SOSTITUZIONE DELLE PAGINE, POPOLARE IL SEGUENTE DIAGRAMMA INDIVIDUANDO L'ALLOCAZIONE DEI FRATTE DOPO OGNI ACCESSO ALLA PAGINA

SOLUZIONE

5	7	7	9	1	5	K	9	9	K	K	K
5	7	7	9	1	5	3	9	9	7	2	5
5	7	7	9	1	5	3	9	9	7	2	5
5	7	7	9	1	5	3	9	9	7	2	5
5	7	7	9	1	5	3	9	9	7	2	5
X	X	V	X	X	V	X	V	V	X	X	X

LRU

*X = PAGE FAULT

*V = PAGE HIT

↓
era il 7

↓
era il 9

↓
era il 5

↓
era il 9

} eravamo quelli che non riempiono i moti de tutto tempo

INDICARE CON DELLE APPOSITE FRECCETTE GLI ISTANTI IN CUI È NECESSARIO INVOCARE L'ALGORITMO DI SOSTITUZIONE DELLE PAGINE

- 5 VOLTE INVOCATO
- 8 PAGE FAULT - 4 PAGE HIT
- FAULT RATE = $8/12 = 0,66 * 100 = 66,66\%$
- HIT RATE = $4/12 = 0,33 * 100 = 33,33\%$

IPOTIZZIAMO UNA ARCHITETTURA CHE FA USO DI PAGINAZIONE IN CUI SI UTILIZZANO OFFSET DI PAGINA 3 Bit E CON UNA MEMORIA CENTRALE DI APPENA 4 FRAME. GLI INDIRIZZI VIRTUALI SONO A 14 Bit. LA TABELLA DELLE PAGINE DI TALE SISTEMA E' DI SEGUITO RIPORTATA:

# PAGINA	# FRAME	# PRESENTE
7	0	0
6	0	0
5	0	1
4	2	1
3	0	0
2	1	1
1	3	1
0	0	0

$2^8 = 256$ Bit (dimensione frame)

1) $\frac{290}{256} = \begin{cases} 1 \text{ pagina} \\ 34 \text{ offset} \end{cases} \rightarrow (3 \cdot 256) + 34 = 768 + 34 = 802$

2) $\frac{797}{256} = \begin{cases} 3 \text{ pagine} \\ 25 \text{ offset} \end{cases} \rightarrow (3 \cdot 256) + 25 = 768 + 25 = 793$

3) $\frac{1025}{256} = \begin{cases} 4 \text{ pagine} \\ 1 \text{ offset} \end{cases} \rightarrow (4 \cdot 256) + 1 = 1024 + 1 = 1025$

UTILIZZARE TALE TABELLA PER EFFETTUARE LA TRADUZIONE DEI SEGUENTI INDIRIZZI VIRTUALI IN FISICI: 290 (100100010), 797 (100011101) E 1025 (10000000001).
RIPORTARE SOLO GLI INDIRIZZI FISICI OTTENUTI (IN BINARIO O DECIMALE) O INDICARE QUALI PORTANO AD UN PAGE FAULT QUANDO E' IL CASO (IGNORARE LA GESTIONE DEL PAGE FAULT STESSO LASCIANDO LA TABELLA INALTERATA)

- 290 (100100010)

11 BIT

- AGGIUNGIAMO 2 BIT ESSENDO STATO DA 9 $\rightarrow$ 00100100010

- NUMERO DI PAGINA $\rightarrow$ 001 (PRIMI 3 BIT) $\rightarrow$ PAGINA 1 $\rightarrow$ FRAME 3 $\rightarrow$ PRESENTE

- 3 FRAME IN BINARIO 11 CHE COMBINATO ALL'OFFSET CI DA 100100010

- 797 (100011101)

INDIRIZZO FISICO

- AGGIUNGIAMO UN BIT $\rightarrow$ 0100011101

- NUMERO DI PAGINA $\rightarrow$ 011 $\rightarrow$ PAGINA 3 $\rightarrow$ FRAME 0 $\rightarrow$ NON PRESENTE

- PAGE FAULT

- 1025 (10000000001)

- NUMERO DI PAGINA $\rightarrow$ 100 $\rightarrow$ PAGINA 4 $\rightarrow$ FRAME 0 $\rightarrow$ PRESENTE

- INDIRIZZO FISICO = 1000000001

IN BIANCO LA RISOLUZIONE PER IL RISULTATO IN DECIMALE

CONSIDERIAMO UN FILE SYSTEM CHE IMPIEGA L'ALLOCAZIONE CON LISTE COLLEGATE.

NOME DI BLOCCO DIM

FRACT.C 233 3056
HELLO.C 123 1036

321	123	200	233	821
0	200	0	821	321
555	288	233	822	212
20	201	532	715	999
555	280	530	50	0
922	399	742	31	843
942	400	221	53	212
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
132	899	343	891	233
135	900	434	882	0

TENENDO CONTO DEL FATTO CHE I BLOCCHI USATI DAL FILE SYSTEM SONO DA 1KB E CHE I NUMERI DI BLOCCO SONO CONTENUTI IN UNA WORD (32 bit); INDIVIDUARE IL VALORE DELLA WORD DI OFFSET 2048 DEL FILE FRACT.C E QUELLO DELL'ULTIMA WORD DEL CONTENUTO DEL FILE HELLO.C.

NOTA: GLI OFFSET SONO ESPRESSI IN BYTE E PARTONO DA 0, ANCHE LA DIMENSIONE DEL FILE NELLA DIRECTORY È ESPRESSA IN BYTE.

SOLUZIONE

FILE-SYSTEM

LISTE COLLEGATE

avremo una lista con i blocchi e il puntatore al blocco successivo

- 1°) PER INDIVIDUARE DOVE SI TROVA L'OFFSET 2048 DOBBIAMO FARE IL SEGUENTE CALCOLO:

$$1024 - 4 \text{ byte} = 1020 \text{ byte} \rightarrow \frac{2048}{1020} = 2,007 \rightarrow 3^{\circ} \text{ BLOCCO}$$

↑
PUNTATORE
SUCCESSIVO

↑
DIR-FILE DEL BLOCCO

1° BLOCCO 233 → 2° BLOCCO 821 → 3° BLOCCO 321

QUINDI L'OFFSET 2048 SI TROVA NEL BLOCCO 321

2°) L'ULTIMA WORD DEL FILE HELLO.C

1° BLOCCO 123 → 2° BLOCCO 200 (ULTIMO BLOCCO)

$$1036 - 4 = 1032 / 1020 = 1,01 = 2$$

GLI INODE UNIX IMPIEGANO INDIRIZZAMENTO DI 12 BLOCCHI DIRETTO E 1 INDIRIZZAMENTO INDIRETTO SINGOLO, DOPO COTRIPLO PER LOCALIZZARE I BLOCCHI DI UN FILE FISICO.

SI SUPPONGA CHE CIASCUN INDIRIZZO OCCUPI 4 BYTE.

SI CALCOLI LA DIMENSIONE DEL PIU' GRANDE FILE CHE PUO' ESSERE GESTITO CON QUESTO SCHEMA D'INDIRIZZAMENTO, PER DIMENSIONI DEI BLOCCHI DI 512, 1024 E 4096 byte

SOLUZIONE

PRIMO BLOCCO 512

- CALCOLIAMO LO SPAZIO DI INDIRIZZAMENTO INDIRETTO $512/4 \text{ byte} = 128$

$$\bullet 12 \cdot 512 = 6144 \text{ byte}$$

$$\bullet 128 \cdot 512 = 65536 \text{ byte}$$

$$\bullet 128^2 \cdot 512 = 8388608 \text{ byte}$$

$$\bullet 128^3 \cdot 512 = 1 \text{ GB}$$

SECONDO BLOCCO 1024

$$- 1024/4 = 256$$

$$\bullet 12 \cdot 1024 = 12288 = 12 \text{ Kb}$$

$$\bullet 256 \cdot 1024 = 256 \text{ Kb}$$

$$\bullet 256^2 \cdot 1024 = 64 \text{ Mb}$$

$$\bullet 256^3 \cdot 1024 = 16 \text{ GB}$$

TERZO BLOCCO 4096

$$- 4096/4 = 1024$$

$$\bullet 12 \cdot 4096 = 48 \text{ Kb}$$

$$\bullet 1024 \cdot 4096 = 4 \text{ Mb}$$

$$\bullet 1024^2 \cdot 4096 = 4 \text{ GB}$$

$$\bullet 1024^3 \cdot 4096 = 4 \text{ Tb}$$

dimensione 12 blocchi diretti + dimensione blocchi indiretti singoli
+ dimensione blocchi indiretti doppi

$$\downarrow$$
$$\left(\text{dimensione blocchi indiretti singoli} \right)^2$$

SUPPONIAMO DI AVERE UN FILE-SYSTEM CHE UTILIZZA PER TENERE TRACCA DEI FILE IN ESSO MEMORIZZATI LA SEGUENTE FAT E CHE PREVEDE LE SEGUENTI VOCI ALL'INTERNO DELLA CARTELLA RADICE:

FAT	
INDICE	BLOCCO
1	4
2	3
3	15
4	5
5	10
6	12
7	1
8	2
9	3
10	1
...	...

CARTELLA RADICE	
NAME	PRIMO BLOCCO
PIPP0.TXT	7

FAT

INDICARE ESATTAMENTE IN QUALE BLOCCO DEL DISCO (INDICARE IL NUMERO DI BLOCCO) E LOCALIZZABILE L'OFFSET 10100 ALL'INTERNO DEL FILE PIPPO.TXT INDICARE QUAL E' LA DIMENSIONE MINIMA PRESUNTA IN BYTE DELLO STESSO FILE. IN TALE CALCOLO TENERE CONTO DEL FATTO CHE UN BLOCCO DEL FILE-SYSTEM E' GRANDE 4KB.
NOTA: GLI OFFSET SONO ESPRESSI IN BYTE E PARTONO DA ZERO

SOLUZIONE

- CALCOLIAMO QUANTI BLOCCHI OCCUPA L'OFFSET

$$\frac{\text{OFFSET}}{\text{DIM. BLOCCO}} = \frac{10100}{4096} = 2,46 \rightarrow 3^{\circ} \text{ BLOCCO}$$

- CALCOLO TERZO BLOCCO:

PRIMO BLOCCO 7 → SECONDO BLOCCO 1 → TERZO BLOCCO 4
L'OFFSET SI TROVA NEL BLOCCO 4 E L'OFFSET SARA':

$$10100 - (4096 * 2) = 1908$$

a noi interessa il terzo blocco quindi calcoliamo i la dim blocco per i primi due blocchi

- DIMENSIONE MINIMA FILE:

$$4 * 4096 + 1 \text{ byte} = 16385 \text{ byte}$$

↑ ↑ ↑
 4 blocchi DIMENSIONE ULTIMO BLOCCO
 pieni Blocco VUOTO

$$M - 1 * \text{DIM BLOCCO} + 1 \text{ BYTE}$$

continuiamo a calcolare i blocchi fino al 10 (-1) e quindi avremo 5 blocchi e facciamo $5 - 1 = 4$ blocchi

IN UN DISCO CON BLOCCHI DI 2Kbyte (= 2^{11} byte), E DEFINITA UN FAT SYSTEM FAT16. OGNI ELEMENTO HA LUNGHEZZA DI 2 byte E INIZIA UN BLOCCO DEL DISCO. LA COPIA PORTANTE DELLA FAT RISIEME UN INDIRIZZO A PARTIRE DAL BLOCCO INDICE 0 E UNA COPIA DI LAVORO VIENE CARICATA IN MEMORIA PRINCIPALE ALL'AVVIAMENTO DEL SISTEMA OPERATIVO. SUPPONENDO CHE LA FAT SIA DIMENSIONATA IL BLOCCO ALLA MASSIMA CAPACITA' IN INDIRIZZABILITA' DEI SUOI ELEMENTI SI CHIEDE:

- 1) IL NUMERO DI BLOCCHI DATI INDIRIZZABILI DALLA FAT
- 2) IL NUMERO DI BYTE E DI BLOCCHI DEL DISCO OCCUPATI DALLA FAT
- 3) L'INDICE DEL PRIMO BLOCCO DATI NEL DISCO
- 4) QUALE CAPACITA' (IN BLOCCHI E IN BYTE DOVE AVERE IL DISCO) PER CONTENERE TUTTI I BLOCCHI DATI INDIRIZZABILI.

SOLUZIONE

FAT

- 1) ESSENDO UNA FAT16 $\rightarrow 2^{16}$
- 2) LA FAT HA 2^{16} ELEMENTI MA SICCOME OGNI ELEMENTO HA LUNGHEZZA DI 2 byte, SARA' $2^{16} \cdot 2 = 2^{17}$. I BLOCCHI OCCUPATI SONO DATI DALLA DIMENSIONE DELLA FAT DIVISO LA DIMENSIONE DEL BLOCCO $2Kb = 2^{11} \cdot 8$
 $= 2^{11} \rightarrow 2^{17} / 2^{11} = 2^6 = 64$ BLOCCHI +2
- 3) IL PRIMO BLOCCO DATI NEL DISCO E' QUELLO DI INDICE 64 (0163 FAT)
- 5) BLOCCHI DATI INDIRIZZABILI 2^{16}
 - DIMENSIONE IN BYTE = 2^{16} BLOCCHI $\cdot 2^{11}$ byte/blocco = 2^{27} byte 27

FACENDO USO DI UNA LISTA CONCATENATA, CHE HA BLOCCHI DA 4Kb
E UN NUMERO DI BLOCCO DI 32 bit, QUANTI BLOCCHI OCCORRONO PER
MEMORIZZARE UN FILE DA 40Kb?

SOLUZIONE

- LA DIMENSIONE DI UN SINGOLO BLOCCO È $4Kb = 4 \cdot 1024 \text{ B} = 4096 \text{ byte}$

- SI DEVE TENERE IN CONSIDERAZIONE LA DIMENSIONE DEL PUNTATORE =

che sarebbe il numero di blocco
 $32/8 = 4 \text{ byte}$ *conversione in byte*

↑
OCI SONO 8 BIT IN UN BYTE

- QUINDI OGNI BLOCCO DELLA LISTA HA UNO SPAZIO PER I DATI DI:

$$4096 - 4 = 4092$$

- IL FILE HA DIMENSIONI: $40Kb = 40 \cdot 1024 = 40960 \text{ byte}$

- I BLOCCHI NECESSARI PER MEMORIZZARE IL SEGUENTE FILE SONO:

$$\frac{\text{DIMENSIONE FILE}}{\text{DIM BLOCCO DATI}} = \frac{40960}{4092} = 10,01$$

- SONO SERVITI POCO PIÙ DI 10 BLOCCHI, CIOÈ 11 BLOCCHI